> A complete deep-dive into two powerful data structures and the modern ES6+ object syntax features that make JavaScript code cleaner and more expressive.

Part 1 — Set

What is a Set?

A `Set` is a **collection of unique values** — it automatically removes duplicates. Order of insertion is preserved.

```
const set = new Set();
console.log(set); // Set(0) {}

const set = new Set([1, 2, 3, 4, 4, 5, 6, 2, 3]);
console.log(set); // Set(6) {1, 2, 3, 4, 5, 6} — duplicates removed
```

Sets Accept Any Iterable

```
// From a string
const set = new Set("Hello");
console.log(set); // Set(4) {'H', 'e', 'l', 'o'} — duplicate 'l'
removed
```

Type-Strict Uniqueness

Sets use strict equality (`===`) to determine uniqueness:

```
const set = new Set([1, '1', true, null, undefined, NaN, NaN]);
console.log(set); // Set(6) {1, '1', true, null, undefined, NaN}
// 1 and '1' are different (number vs string)
// NaN is treated as equal to itself (unique behavior — NaN !== NaN
normally!)
```

> **Special case:** In regular JS, `NaN !== NaN`. But inside a Set, two `NaN` values are considered equal — only one is stored.

Set Methods

```
const set = new Set([1, 2, 3, 4]);

// has() — check if a value exists → boolean
console.log(set.has(2)); // true
console.log(set.has(5)); // false

// add() — add a value (ignored if already exists), returns the Set
set.add(50);
set.add(50); // ignored — 50 already exists
set.add(50).add(60).add(70); // chainable!
console.log(set); // Set(7) {1, 2, 3, 4, 50, 60, 70}

// delete() — remove a value → true if existed, false if not
console.log(set.delete(2));  // true
console.log(set.delete(99)); // false
console.log(set); // Set(6) {1, 3, 4, 50, 60, 70}

// clear() — remove all values
set.clear();
console.log(set); // Set(0) {}

// size — number of unique elements (property, not method)
const set2 = new Set([1, 2, 3, 4, 11]);
console.log(set2.size); // 5
```

Use Case — Remove Duplicates from Array

```
const arr = ['banana', 'orange', 'apple', 'orange', 'banana'];

// Spread Set back into array
const uniqueItems = [...new Set(arr)];
console.log(uniqueItems); // ['banana', 'orange', 'apple']
```

This is the most common real-world use of Set — a one-liner deduplication.

Set Operations (Math)

Union — All elements from both sets

```
const setA = new Set([1, 2, 3]);
const setB = new Set([3, 4, 5]);

// Modern (ES2025+)
const union = setA.union(setB);
console.log(union); // Set(5) {1, 2, 3, 4, 5}

// Compatible alternative
const union = new Set([ ...setA,  ...setB]);
console.log([ ...union]); // [1, 2, 3, 4, 5]
```

Intersection — Elements in BOTH sets

```
const setA = new Set([1, 2, 3]);
const setB = new Set([3, 4, 5]);

// Modern
console.log(setA.intersection(setB)); // Set(1) {3}

// Alternative
const intersection = new Set([ ...setA].filter(x => setB.has(x)));
```

Difference — Elements in A but NOT in B

```
const setA = new Set([1, 2, 3]);
const setB = new Set([3, 4, 5]);

console.log(setA.difference(setB)); // Set(2) {1, 2}   - in A, not in
B
console.log(setB.difference(setA)); // Set(2) {4, 5}   - in B, not in
A
```

Symmetric Difference — Elements in either but NOT in both

```
console.log(setA.symmetricDifference(setB)); // Set(4) {1, 2, 4, 5}
// Everything except what they share (3)
```

Visual Summary of Set Operations

```
setA = {1, 2, 3}
setB = {3, 4, 5}

Union:                {1, 2, 3, 4, 5}   — everything
Intersection:         {3}               — only shared
A.difference(B):      {1, 2}            — A minus shared
B.difference(A):      {4, 5}            — B minus shared
SymmetricDifference: {1, 2, 4, 5}       — everything except shared
```

Iterating Over a Set

```
const setB = new Set([3, 4, 5]);

for (const value of setB) {
  console.log(value); // 3, 4, 5
}
```

Part 2 — Map

What is a Map?

A `Map` is a **collection of key-value pairs** where **keys can be any type** — objects, booleans, functions, anything.

This is the critical difference from regular objects, where all keys are converted to strings.

```
const map = new Map([
  ["name", "mona"],       // string key
```

```
  [true, 'value2'],       // boolean key
  [{ id: 1 }, 'object']  // object key
]);

console.log(map);
```

Maps are created from an **array of `[key, value]` pairs** (entries).

Why Map Over Regular Objects for Keys?

```
// Regular object - ALL keys become strings
const user = {};
user[true]   = 1;
user["true"] = 40;

console.log(user);        // { true: 40 } - only ONE entry! both
became "true"
console.log(user[true]);  // 40
console.log(user["true"]); // 40 - same key!

// Map - keys preserve their type
const map = new Map();
map.set(true,   "boolean value");
map.set("true", "string value");

console.log(map.get(true));   // "boolean value"
console.log(map.get("true")); // "string value" - completely
different entries!
```

Map Methods

```
const obj = { id: 1 };
const map = new Map([
  ["name", "mona"],
  [true, 'value2'],
  [obj, 'object']
```

```
]);

// get() – retrieve by key
console.log(map.get("name")); // "mona"
console.log(map.get(true));   // "value2"
console.log(map.get("true")); // undefined – different key!

// Object keys use reference equality:
console.log(map.get({ id: 1 })); // undefined – different object
reference!
console.log(map.get(obj));       // "object" – same reference ✓

// set() – add or update a key-value pair, returns the Map
map.set("email", "mona@gmail.com");
map.set(true, "updated value"); // overwrites the existing true key

// has() – check if key exists → boolean
console.log(map.has("name")); // true
console.log(map.has("age"));  // false

// delete() – remove by key → boolean
map.delete("name");

// size – number of entries
console.log(map.size); // 3

// clear() – remove all entries
map.clear();
```

> **Reference equality for object keys:** Two objects `{id:1}` and `{id:1}` are different references in memory. Only the exact same variable reference will match as a Map key.

Iterating Over a Map

```
const obj = { id: 1 };
const map = new Map([
  ["name", "mona"],
```

```
  [true, 'value2'],
  [obj, 'object']
]);

// Destructuring in for...of
for (const [key, value] of map) {
  console.log(key, value);
}
// name mona
// true value2
// {id:1} object
```

Map ↔ Array Conversion

```
const map = new Map([["name", "mona"], [true, 'value2'], [obj,
'object']]);

// Map → flat array
console.log([ ...map].flat()); // ["name", "mona", true, "value2",
{id:1}, "object"]
```

Map vs Object — When to Use Which

	Object	Map
Key types	Strings and Symbols only	Any type
Key order	Not guaranteed (integers sorted)	Insertion order always
Size	`Object.keys(obj).length`	`map.size`
Iteration	Needs `Object.keys/values/entries`	Directly iterable
Performance	Good for static structures	Better for frequent add/delete
JSON support	`JSON.stringify()` works	Needs conversion first

	Object	Map
Use when	Data structure, config, records	Dynamic key-value lookup

---

Part 3 — Modern Object Syntax

Property Shorthand

When a variable name matches the property name, you can omit the value:

```
const name = "omar";
const age = 30;

// Old way
const user = { name: name, age: age };

// Shorthand
const user = { name, age };
// Same result: { name: "omar", age: 30 }
```

Computed Property Names `[]`

Use a variable as a property key by wrapping it in `[]`:

```
// Without computed props — key is literally the string "key"
const key = "name";
const user = { key: "omar" };
console.log(user); // { key: "omar" } — NOT what we wanted!

// With computed props — key's VALUE is used as the key
const user = { [key]: "omar" };
console.log(user); // { name: "omar" } ✓

// Any expression works inside []
const prefix = "user";
const user = {
```

```
  [prefix + "Id"]:   1,      // userId: 1
  [prefix + "Name"]: "mona"  // userName: "mona"
};
console.log(user); // { userId: 1, userName: "mona" }
```

Method Shorthand

```
// Old way
const user = {
  greet: function() {
    console.log("Hello");
  }
};

// Method shorthand — cleaner
const user = {
  greet() {
    console.log("Hello");
  },
  greet2: () => {
    // Arrow function method — no own 'this'
  }
};
```

Part 4 — Object Static Methods

`Object.keys()` — Get All Keys

```
const user = { name: "omar", age: 30 };

const keys = Object.keys(user);
console.log(keys); // ["name", "age"]

// Useful for iteration when you need the key
for (let i = 0; i < keys.length; i++) {
  console.log(keys[i], user[keys[i]]); // name omar / age 30
```

```
}

// Check if object is empty
console.log(keys.length === 0); // false

// Works on arrays too
const arr = ["mona", "ali", "omar"];
console.log(Object.keys(arr)); // ["0", "1", "2"] — indices as
strings!
```

`Object.values()` — Get All Values

```
const user = { name: "omar", age: 30 };
console.log(Object.values(user)); // ["omar", 30]

// Practical — find max price
const prices = { apple: 50, banana: 20, orange: 10 };
console.log(Math.max(...Object.values(prices))); // 50

// Filter stock
const stock = { phone: 10, laptop: 5, smartWatch: 0, airpods: 0 };
const inStock = Object.values(stock).filter(count => count > 0);
console.log(inStock); // [10, 5]
```

`Object.entries()` — Get All `[key, value]` Pairs

```
const user = { name: "omar", age: 20, email: "omar@gmail.com" };

console.log(Object.entries(user));
// [["name", "omar"], ["age", 20], ["email", "omar@gmail.com"]]

// Iterate with destructuring
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

```
// Transform — wrap each entry back into its own object
const result = Object.entries(user).map(([key, value]) => ({ [key]:
value }));
// [{name:"omar"}, {age:20}, {email:"omar@gmail.com"}]
```

Object ↔ Map Conversion

```
const user = { name: "omar", age: 20, email: "omar@gmail.com" };

// Object → Map
const map = new Map(Object.entries(user));
console.log(map); // Map(3) { "name" => "omar", "age" => 20, "email"
=> "..." }

// Map → Object (Object.fromEntries)
const entries = [['name', 'mona'], ['age', 30], [true, 'value3']];
console.log(Object.fromEntries(entries));
// { name: "mona", age: 30, true: "value3" }

const map2 = new Map([['name', 'mona'], ['age', 30]]);
console.log(Object.fromEntries([...map2]));
// { name: "mona", age: 30 }
```

`Object.assign()` — Copy and Merge Objects

Copies all properties from source objects into a target object. Later sources override earlier ones.

```
const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, b: 30 }; // b will override obj1's b
const obj3 = { d: 5 };

// Object.assign(target, source1, source2, ...)
// MUTATES the target!
console.log(Object.assign(obj1, obj2, obj3));
// { a: 1, b: 30, c: 3, d: 5 }
```

```
console.log(obj1); // { a: 1, b: 30, c: 3, d: 5 } — obj1 was mutated!

// Safe — use empty {} as target to avoid mutation
console.log(Object.assign({}, obj1, obj2, obj3));
console.log(obj1); // { a: 1, b: 2 } — unchanged ✓
```

`Object.assign()` vs Spread

```
// These are equivalent:
const merged1 = Object.assign({}, obj1, obj2, { a: 40 });
const merged2 = { ...obj1, ...obj2, a: 40 };

// Spread is preferred in modern code — cleaner and more readable
```

Real-World Pattern — Default Settings with `Object.assign()`

```
function setUserSettings( settings ) {
  const defaultSettings = {
    theme: "light",
    language: "en",
    notifications: false
  };

  return Object.assign({}, defaultSettings, settings);
  // settings properties override defaults, missing ones use defaults
}

console.log(setUserSettings({ theme: "dark" }));
// { theme: "dark", language: "en", notifications: false }

console.log(setUserSettings({ theme: "dark", language: "ar" }));
// { theme: "dark", language: "ar", notifications: false }

console.log(setUserSettings());
// TypeError! settings is undefined — need to handle this:
```

Better approach — destructuring with defaults in parameters:

```
function setUserSettings({ theme = 'light', language = 'en',
notifications = false } = {}) {
  return { theme, language, notifications };
}

console.log(setUserSettings({ theme: "dark" }));
// { theme: "dark", language: "en", notifications: false }

console.log(setUserSettings()); // works! the = {} handles missing
argument
// { theme: "light", language: "en", notifications: false }
```

> Both approaches work. Destructuring with defaults is more declarative. `Object.assign()` is useful when you need to merge dynamically at runtime.

Complete Summary

```
Set:
  - Unique values only, any type
  - Methods: add, has, delete, clear, size
  - Operations: union, intersection, difference, symmetricDifference
  - Best for: deduplication, membership testing, math set operations

Map:
  - Key-value pairs, keys can be ANY type
  - Object keys use reference equality
  - Methods: set, get, has, delete, clear, size
  - Best for: dynamic lookups with non-string keys

Modern Object Syntax:
  - Property shorthand:    { name }  →  { name: name }
  - Computed keys:         { [key]: value }
  - Method shorthand:      greet() {} instead of greet: function() {}

Object Static Methods:
  - Object.keys(obj)      →  array of keys
  - Object.values(obj)    →  array of values
  - Object.entries(obj)   →  array of [key, value] pairs
```

```
- Object.fromEntries()  →  entries array/map → object
- Object.assign()       →  merge objects (mutates target!)
```

Abdullah Ali

Contact : +201012613453